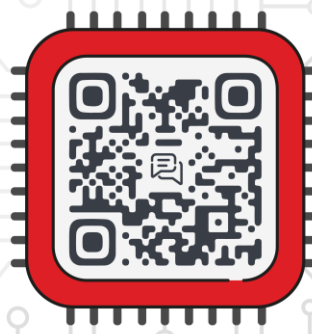




5

24

2880 sp

نظري

كلية الهندسة المعلوماتية

السنة الثانية

Java collection
framework

د. محمد الأحمد



RB Informatics; 20/11/2024

برمجة ٣

السلام عليكم

هذه المحاضرة تشمل ما سنتعلمه في مادتي الخوارزميات 1 و 2 ، وسنتعلم data structure تتضمن توابع جاهزة في الجافا يجب فقط تعلم كيفية استخدامها.

- Data structure: how to organize data in memory (stack, queue).

تمتلك تمثيلاً رياضياً يبين بنيتها و كذلك عمليات يمكن تنفيذها عليها.

مثلاً: array تمثل متسلسلة رياضياً يمكن أن ننفذ عليها عمليات كعملية البحث عن عنصر وعملية الوصول لترتيب معين وهكذا...

- تعرفنا سابقاً على primitive data type (int, float) والتي تخزن قيمة واحدة بمكان محدد بالذاكرة، والآن سنتعلم عدة طرق لتخزين عدة elements من الـ data في الـ memory حسب الحاجة في المسألة، ونميز نوعين:

1. Linear Data Structures : بنى معطيات خطية، سميت بذلك لأن الـ data تحتل فيها بعداً واحداً في الذاكرة، منها: الـ stack, queue, array, linked list.

متى نستخدم بنى المعطيات الخطية؟

عندما تكون البيانات في المسألة التي نحاول حلها لا تشتمل علاقات معقدة تمثل علاقات هرمية أو علاقات نقطية متشعبة، فيكفي تخزينها في بنى خطية. نختار البنية الخطية بناءً على المطلوب منا ضمن المسألة:

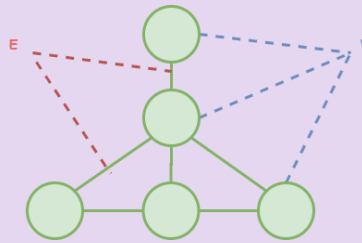
- فمثلاً إذا كان لدينا data ويهملنا فقط الوصول لها بتعقيد $O(1)$ نستخدم array.
- أما إذا كان لدينا data ونريد تطبيق عمليات حذف وإضافة كثيرة عليها في مسألتنا نقوم بالتخزين في بنية الـ linked list
- أما لو أردنا أن تكون عملية الإضافة والحذف من نفس الجهة نستخدم الـ stack ، كما الأطباق عندما نضعها فوق بعضها ونريد أخذ طبق سنأخذ آخر طبق وضعناه في أعلى الأطباق، وهذا ما يدعى last in first out (آخر عنصر ندخله هو أول عنصر نخرجه - آخر طبق نضعه فوق الأطباق هو أول طبق نأخذه).

- أما لو أردنا أن تكون عملية الإضافة من جهة والحذف من جهة أخرى نستخدم ال queue ، كما الطابور عندما يأتي شخص ندخله من جهة، و عندما نخرج شخصاً من الطابور نخرجه من جهة أخرى، وهذا يسمى first in first out (أول عنصر ندخله هو أول عنصر نخرجه - أول شخص دخل الطابور هو أول شخص يخرج منه).

2. Non linear DS : بنى معطيات غير خطية مثل Hash table, graph, tree وهذا النوع من ال Ds

سندرسه في الفصل الثاني

- ال graph (البيان) تتكون من عقد مرتبطة بوصلات وتكون اما موجهة او غير موجهة وتستخدم لتمثيل : الطرقات في المدن أو خريطة أو مواقع تواصل اجتماعي بشكل أعم أي شيء يمكن تمثيله بمجموعة حالات، ومجموعة انتقالات بين هذه الحالات يمكننا اعتبارها غراف، مثل الشطرنج ، XO وتمثل بالشكل:



- تمثل ال array بخط مستقيم

- تمثل ال linked list بعقد وكل عقدة تؤشر على التي تليها

ملاحظة : أهم الإجراءات في :

- ال graph : زيارة العقدة ونقوم بتعليمها أنها مزارة، و زيارة العقد المجاورة لها (إيجاد طول ال path و المسار) في ال graph ، لن نتعمق كثيراً بها لأننا سندرسها في الخوارزميات (2)
- ال tree : زيارة عقدة / إضافة عقدة أو حذفها
- ال stack : Push : تستخدم من أجل إضافة عناصر الى ال stack وتكون الإضافة من جهة واحدة فقط
- Pop : تستخدم لحذف اخر عنصر قمنا بإضافته الى ال stack
- Top : تستخدم من أجل عرض اخر عنصر قمنا بإضافته الى ال stack

إطار تجميع جافا java collection framework

- هو مجموعة من الهياكل وال Interfaces التي توفر طرقاً متقدمة للتعامل مع collection.
- الكائنات في جافا تعرف الإطار كبنية موحدة لتمثيل ال collection ومعالجة المجموعات بشكل مستقل عن تفاصيل التنفيذ.

(Unified architecture for representing and manipulating collections enables these collections to be handled independently of their implementation details)

- يعطي الإطار واجهات وخوارزميات سهلة الاستخدام للتعامل مع بنى معطيات مهما كان نوعها
(This framework includes interfaces, implementations, and algorithms that provide a range of useful data structures and methods for managing collections.)

بناء ال collection في جافا :

- كل فكرة أساسية فيه تمثل ب انترفيس
- كل طريقة يمكن استخدامها لتنفيذ هذه الفكرة تمثل بكلاس (كلاسات + خوارزميات)

: A collections framework contains three things

- Interfaces
- Implementations
- Algorithms

ملاحظات :

- المسألة تفرض علينا نوع ال DS التي نستخدمها بحيث تكون ال operation التي نطبقها سهلة وسريعة
- قبل java1.2 كان لدينا كلاسين فقط هما vector و stack واستخدمنا توابع ال push للضافة، و pop للحذف، ولكن بعد إصدار ال collections framework -وهي عبارة عن انترفيسات عامة بحيث نستخدم الأبناء وال methods التي بداخلها بعد أن نقوم ب implements لها- أصبح لدينا methods بطريقة موحدة لكافة ال DS مثل ال add, reverse.
- جميع ال collection تتعامل مع reference ، ممنوع التعامل مع primitive data type
(1 مثل array تتعامل مع primitive data type ، أما ال ArrayList فهي تتعامل مع object)
- Collection الأعداد الصحيحة حيث كل عدد int هو reference من نمط Integer
(2 جميع الدوال التي سنتطرق لدراستها تتعامل مع object أي يجب أن ننتبه لعملية ال handling ل NullPointerException (أي وضع try , catch).
- collection هو عبارة عن كائن object يحوي بداخله على مجموعة أيضا من الكائنات وهو أيضا تجمع interfaces عامة تمثل جميع ال DS.

(collection is an object that groups multiple elements into a single unit and represents a group of objects)

- جميع الـ methods في الـ collection ينبغي أن تنفذ فكرة معينة ولكن بغض النظر عن آلية تطبيقها (The manipulation should be independent of the implementation details)

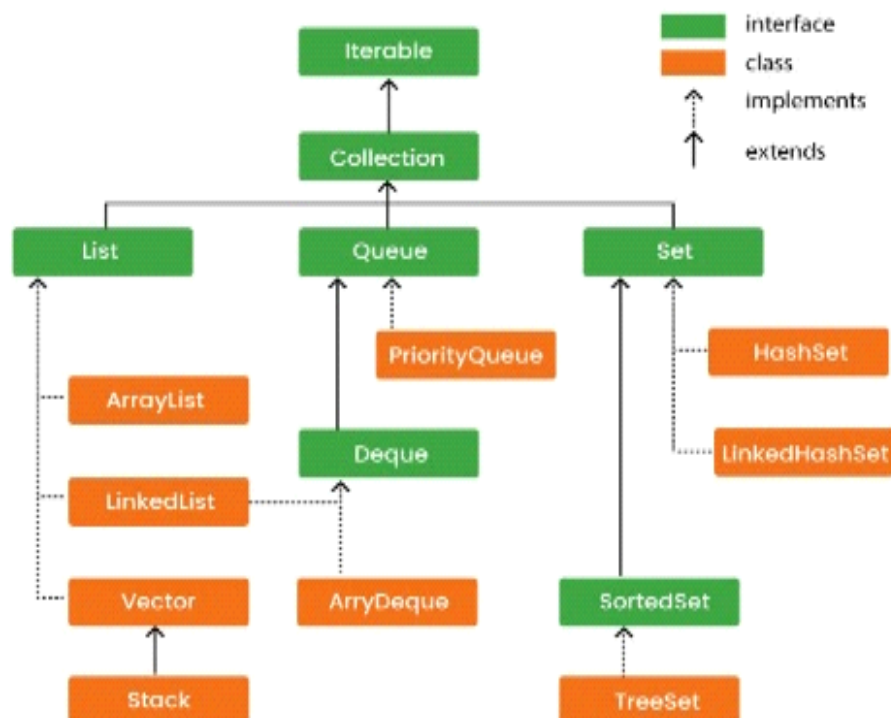
مميزات Java Collections :

1. Very useful
تفيد في تحسين الأداء
2. store, retrieve and manipulate data
تساعد في تخزين البيانات في الذاكرة أو في قاعدة بيانات بحيث يمكن الوصول إليها لاحقاً.
3. transmit data from one method to another
4. استرجاع واستعادة المعلومات المخزنة عند الحاجة
5. data structures and methods written by hotshots in the field
6. معالجة البيانات أي إجراء عمليات معينة على البيانات مثل التعديل أو الإضافة أو الحذف

ملاحظة :

- جميع مخططات الوراثة بين الكلاسات التي مرت معنا يجب فهمها وحفظها
- نستطيع استدعاء الـ collection أو الـ map من الحزمة java.util.

Collection Interface :



- تعتبر جزءاً من ال collection framework حيث تقدم طريقة موحدة للتعامل مع collection الكائنات
- الانترفيس Collection مصمم بشكل عام لاحتواء جميع الأشياء المشتركة في جميع ال Collection وهو الأساس لجميع ال Collections ، وفي حال تم استدعاء أي دالة منه بشكل خاطئ سيتم رمي UnsupportedOperationException
- ترث ال collection interface من ال Iterator interface

تعريف ال collection :

```
Collection<String /float... > myCollection = new DataStructureType<>();
```

ملاحظات :

- Java allows using :
 - 1) Lists
 - 2) Sets
 - 3) Hash tables (maps)
 - Collection Interface defines fundamental methods that are sufficient to establish the basic behavior of a collectionint:
 - int size(); ترجع عدد العناصر في المجموعة
 - boolean isEmpty(); تفحص إذا كانت المجموعة خالية من العناصر أم لا
 - boolean contains(Object element); ترجع true إذا كان العنصر المحدد موجوداً
 - boolean add(Object element); تضيف العنصر المحدد إلى المجموعة
 - boolean remove(Object element); تزيل نسخة واحدة من العنصر المحدد في المجموعة
 - Iterator iterator(); ترجع تكرار العناصر في المجموعة
- These methods are enough to define the basic behavior of a collection
- Collection Interface provides an Iterator to step through the elements in the Collection

ملاحظة : لا ننسى أن جميع الباراميترات هي عبارة عن كائنات

Iterator interface :

هي واجهة تتيح لنا انشاء كائنات يمكن تكرارها باستخدام حلقة for each وهي جزء من java.lang وتوفر وسيلة موحدة للتكرار عبر مجموعة من العناصر.

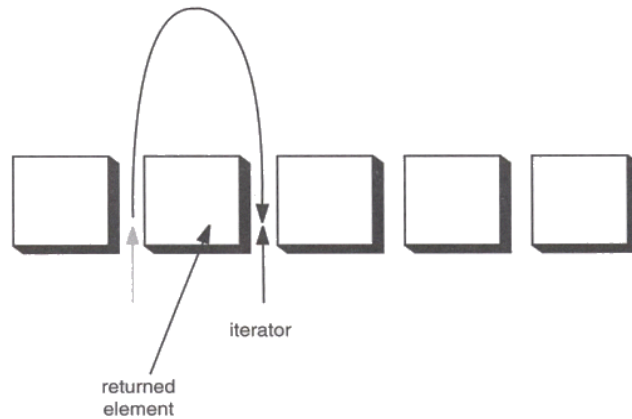
تحتوي هذه الواجهة على دالة iterator والتي ترجع كائن من نوع iterator يمكن استخدامه للتنقل عبر العناصر وتحتوي على ثلاثة methods أساسية توفر ميزة الوصول إلى محتويات المجموعة (provide access to the contents of the collection)

Object next(): ترجع العنصر التالي الموجود في الكائن وترجعه ك object
 أي عندما نستدعي الدالة next() فإنها تقوم باسترجاع العنصر الحالي في المجموعة ثم ينتقل
 الـ Iterator إلى العنصر التالي في المجموعة وبعدها يمكننا القيام بأي عملية أخرى كحذفه أو أي
 شيء آخر

(Each call to next() 'reads' an element from the collection, which you can then use or remove)

boolean hasNext(): ترجع (true) في حال كان كائن الـ Iterator لا يزال يحتوي على عناصر

void remove(): تحذف العنصر الحالي الموجود في الكائن الذي أرجعته الدالة next() قبل
 استدعائها



طريقة استخدام الـ Iterator :

- 1- جلب عناصر أي كائن بواسطة الدالة Iterator وتخزينهم مباشرة في كائن من الـ Iterator
- 2- انشاء حلقة تمر على العناصر الموجودة في كائن الـ Iterator باستخدام الدالة hasNext()
- 3- الحصول على عنصر واحد من العناصر الموجودة في الكائن Iterator بواسطة الدالة next()

```
1 import java.util.Collection;
2 import java.util.*;
3 public class Main {
4     public static void main(String[] args) {
5         Collection c;
6         c = new ArrayList();
7         System.out.println(c.getClass().getName());
8         for (int i=1; i <= 10; i++) {
9             c.add(i + " * " + i + " = " + i*i);
10        }
11        Iterator iter = c.iterator();
12        while (iter.hasNext())
13            System.out.println(iter.next());
14    }}
```

مثال باستخدام الـ Iterator :

الشرح :

- تعريف كائن Collection من نوع ArrayList وطباعة اسم الكلاس لهذا الكائن.
- يتم استخدام حلقة for لإضافة عناصر إلى المجموعة c كل عنصر هو سلسلة نصية تمثل معادلة ضرب.
- يتم إنشاء كائن Iterator للمجموعة c، واستخدام حلقة while لطباعة كل عنصر في المجموعة باستخدام next() التي تنتقل من عنصر إلى العنصر التالي.

Queue interface :

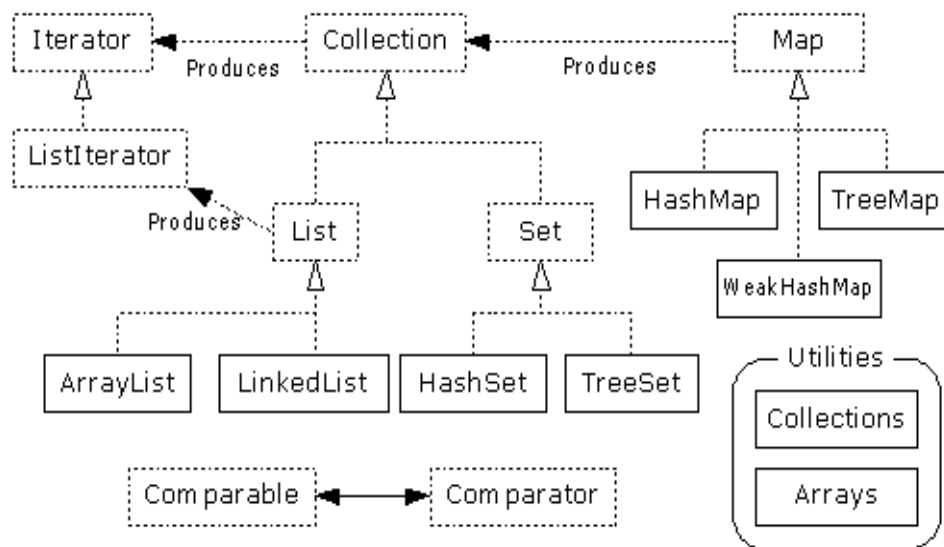
تمثل جزءا من collection framework وهي تمثل هيكلية البيانات وتستخدم لتنظيم العناصر وفق مبدأ الـ FIFO (First In First Out) حيث أول عنصر يتم إضافته هو أول عنصر يتم إخراجها

Offer	تستخدم لإضافة عنصر الى نهاية الـ Queue
Poll	تستخدم لاستخراج وحذف العنصر من بداية الـ Queue
Add	تستخدم لإضافة عنصر الى بداية Queue
Peek	تستخدم للحصول على العنصر الذي يوجد في بداية الـ Queue دون حذفه

فائدة :

Stack interface : تمثل جزءا من collection framework وهي تمثل هيكلية البيانات وتستخدم لتنظيم العناصر وفق مبدأ الـ LIFO (Last In First Out) حيث أول عنصر يتم إضافته هو آخر عنصر يتم إخراجها

Set interface :



تستخدم لتمثيل مجموعة من العناصر بحيث يكون التكرار غير مسموح ولا أهمية للترتيب فمثلاً المجموعة (1,2) نفسها المجموعة (2,1) لأن الترتيب غير مهم (كل عنصر يملك قيمة مختلفة)

تعريف الـ set :

```
Set < String /float... > myset= new SetType<>();
```

حيث SetType من الممكن أن يكون HashSet أو TreeSet أو غيرها.

جميع توابع الـ collection هي نفسها توابع الـ set (Same methods as Collection)

لا تضمن Set الأساسية أي ترتيب للعناصر، ولكن يمكن استخدام كلاسات فرعية مثل TreeSet للحصول على ترتيب معين عند الحاجة (No guaranteed order in the basic Set interface)، أي ان الـ Set مصممة تصميماً مختلفاً وتعمل بطريقة محددة تمنع التكرار (different contract - no duplicate entries) لديها دالتان توفران وظائفها الأساسية (Defines two fundamental methods) وهما:

boolean add (Object o)	تضيف كائناً جديداً في الكائن الذي قام باستدائها - ترجع true اذا تمت عملية الإضافة بنجاح - reject duplicates.
Iterator iterator()	ترجع كائناً نوعه iterator يحتوي على جميع عناصر الكائن الذي قام باستدائها، حيث توفر كائناً من نوع Iterator (أداة للتكرار) يسمح لك بالتنقل عبر العناصر الموجودة في المجموعة (Set) عنصراً تلو الآخر. (Provides an Iterator to step through the elements in the Set)
void Clear()	لإزالة جميع العناصر من المجموعة مما يجعلها فارغة

No guaranteed order in the basic Set interface

There is a SortedSet interface that extends Set

الفرق بين:

List	Set
يمكن وجود نفس القيمة في أكثر من عنصر	لا يمكن وجود نفس القيمة في أكثر من عنصر

فائدة :

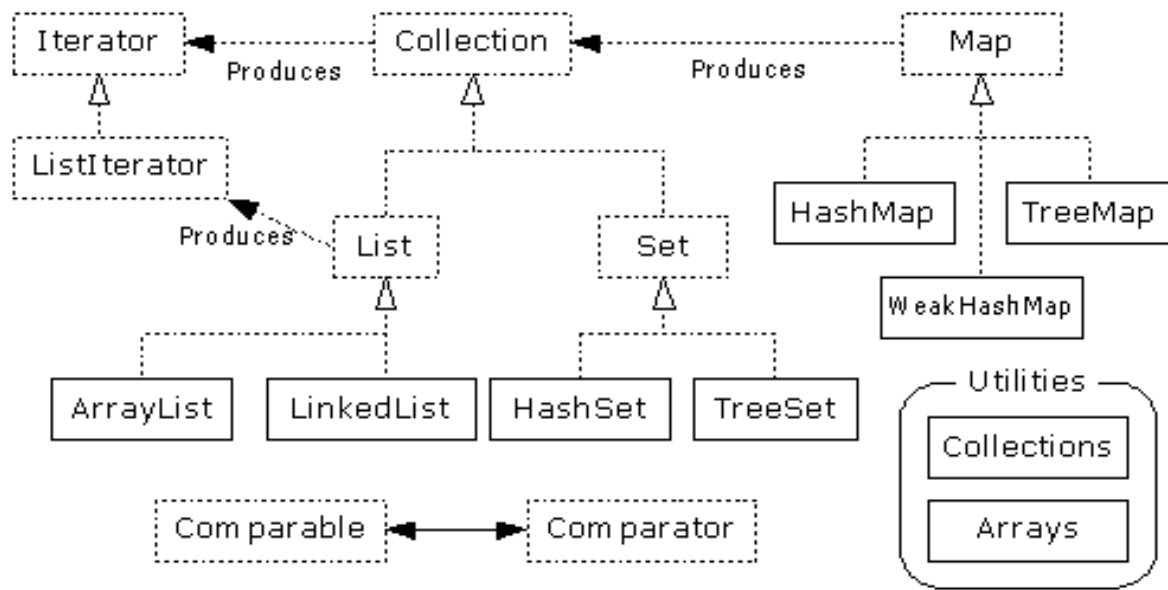
: Hashset and Treetset context

1. Hashset :

- يستخدم لتخزين عناصر متتالية ويسمح بتخزين أكثر من عنصر ولكن ليس لهم نفس القيمة
- لا يمكن الوصول الى العناصر في الـ HashSet لأنها لا تملك index خاص بها
- يستخدم هذا الكلاس بسبب سرعته في التخزين مقارنة مع باقي الكلاسات في set
- كل عنصر يضاف في Hashset يملك hashCode خاص به في الذاكرة
- HashCode هي دالة تقوم بتحويل البيانات (نصوص او ارقام) الى قيمة عددية صحيحة مما يسهل عملية البحث والتخزين، واذا كان لدينا كائنان مختلفان يجب ان يكون لهما hashCode مختلف، واذا تم توليد ذات الـ hashcod لهما سيتسبب ذلك بحدوث التصادم (collision).

2. Treetset :

- أحد الكلاسات التي تنفذ Set interface والتي تمثل مجموعة من العناصر الفريدة أي أنها لا تسمح بتكرار العناصر، و يقوم الكلاس Treetset بتخزين العناصر في ترتيب معين (عادةً تصاعدي) بناءً على ترتيب العناصر.



- ملاحظة أريسيزية: sorted interface : sortedset يرث من الانترفيس Set تم بناؤه لتخزين العناصر بترتيب تصاعدي (في حال كانت العناصر ارقام يتم ترتيبهم من الأصغر الى الأكبر)

public Object first()	ترجع قيمة اول عنصر موجود في الكائن الذي قام باستدائها (قيمتها اصغر قيمة لان العناصر مرتبة)
public Object last()	ترجع قيمة اخر عنصر موجود في الكائن الذي قام باستدائها (قيمتها اكبر قيمة لان العناصر مرتبة)
public sortedset headset (object end)	ترجع كائن نوعه sortedset يحتوي على جميع عناصر الكائن الذي قام باستدائها الموجود قبل الكائن end

list interface :

هي مجموعة مرتبة (ordered collection) تسمح بالعناصر المكررة وتحافظ على الترتيب الذي يتم إدراج العناصر به. يوفر طرقا للوصول إلى العناصر وإدراجها وإزالتها بناء على مواضع ال index الخاصة بها. يرث من ال Collection interface ويمتلك دواله الخاصة بالإضافة الى الدوال التي ورثها

The list interface adds the notion of order to a collection

The user of a list has control over where an element is added in the collection

Lists typically allow duplicate elements

Provides a ListIterator to step through the elements in the list.

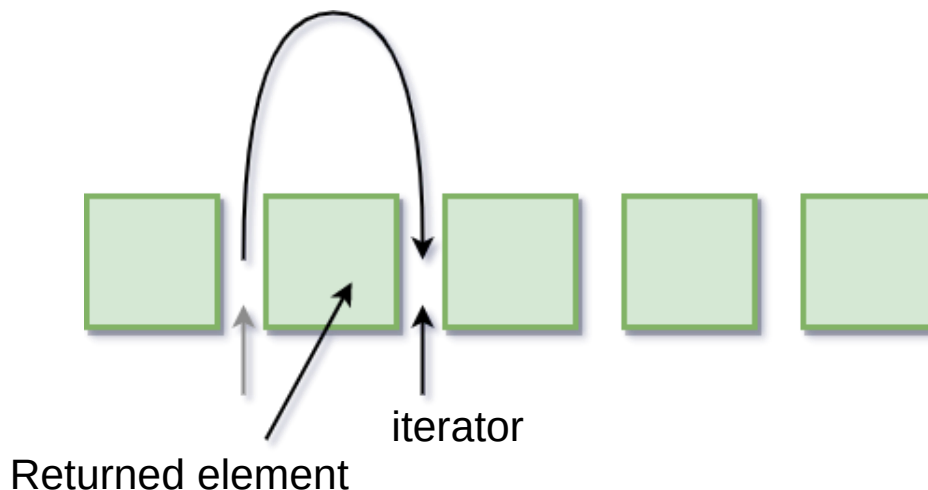
■ تعريف ال list :

```
List< String /float... > names = new ArrayList<>();
```

سنستعرض بعض الدوال ولكن هناك العديد غيرها

Set(int index, Element e)	تبدل قيمة عنصر محدد من عناصر الكائن الذي قام باستدائها بقيمة جديدة
Get (int index)	ترجع الكائن الموجود في ال index الذي نمرره لها
indexOf(Element e)	تبحث في الكائن الذي قام باستدائها عن index العنصر الذي نمرره لها وفي حال لم يكن موجودا تعيد -1

List iterator interface



- Extends the Iterator interface أي يرث من الـ Iterator ويملك دوال إضافية تمكننا من المرور على العناصر الموجودة فيه من الأول الى الأخير او العكس
- تعريف ال List iterator مثل ال iterator (A ListIterator knows position within list)

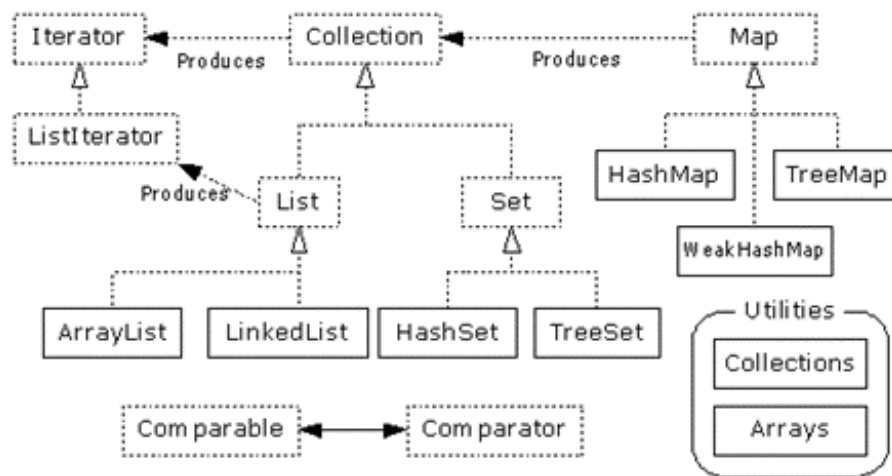
Defines three fundamental methods (3 توابع إضافية عن ال iterator وهي):

void add(Object o) before current position	تضيف الكائن الذي نمرره لها ك في الكائن الذي قام باستدعائها
boolean hasPrevious()	ترجع true في حال الكائن يملك عنصر او اكثر قبل العنصر التالي
Object previous()	ترجع العنصر السابق الموجود في كائن List Iterator ترجعه ك (Object)

ملاحظة ارييسيزية: اذا قمنا باستدعاء التابع next قبل استدعاء التابع add عندها سيتم إضافة الكائن بعد اخر كائن ارجعته الدالة next

Array List and LinkedList Context

ArrayList و LinkedList هما كلاسان ينفذان (implement) ال list interface



: ArrayList (A

- الكلاس ArrayList يرث من الكلاس Abstract list ويطبق الانترفيس list
- اهم ما يميز ال Array List ان عدد العناصر غير ثابت فمن الممكن ان يزداد عدد العناصر عند الإضافة او ينقص عند الحذف بشكل تلقائي وهذا غير ممكن في ال array العادية
- ArrayList تدعم إضافة العناصر أو إزالتها بناءً على index محدد ولكن قد تكون هذه العملية مكلفة (Indexed add and remove are provided, but can be costly if used frequently)
- من الأفضل تغيير حجم Array List (الذاكرة المخصصة) دفعة واحدة بدل الاعتماد على التوسيع التلقائي الذي يتم تدريجياً يتم نسخ العناصر الحالية إلى مساحة جديدة في كل مرة يتم فيها التوسيع مما يؤدي الى عمليات توسعة متعددة للذاكرة

The indexed get and set methods of the List interface are appropriate to use since
ArrayLists are backed by an array

: Methods in ArrayList

Object remove(int index)	تستخدم لحذف العنصر الموجود على ال index الذي نمرره لها وقد يرمي في حال كان ال index فيها اصغر من او اكبر من عدد العناصر الموجودة
Object get(int index)	تستخدم لارجاع العنصر الموجود على ال index الذي نمرره لها وقد يرمي في حال كان ال index فيها اصغر من او اكبر من عدد العناصر الموجودة
Object set(int index, Object element)	تستخدم لتعيين قيمة محددة
Void ensure Capacity (int minCapacity)	تحدد عدد العناصر الأقل الذي يجب ان يحجز لكائن ال في الذاكرة اذا كان العدد الموضوع فيها اكبر من عدد العناصر المحجوز لكائن ال Array List عندها يتم تكبير حجمه واذا كان اصغر فلن تفعل شيء
void add(int index, Object element)	حيث ال Object هو الكائن الذي سيضاف وال index هو رقم العنصر الذي سيضاف فيه الكائن تضيف عنصر جديد في مكان محدد في ال ArrayList

الفرق بين Array و Array list

Array List

لها سعة افتراضية وعادة يتم تعيينها ك 10 عناصر وعندما تتجاوز عدد العناصر لهذه السعة يتم زيادتها بشكل ديناميكي
نستخدم الدالة size أيضا لمعرفة عدد عناصرها

Array

لها حد ثابت يتم تحديده عند الانشاء ولا يمكن تغييره
نستخدم الدالة length لمعرفة عدد عناصرها

- يتم تخزين كل عنصر في القائمة في عقدة وهي تحتوي على البيانات data ومؤشر next يشير الى العقدة التالية previous ومؤشر يشير الى العقدة السابقة
(Stores each element in a node and Each node stores a link to the next and previous nodes)
- تعتبر عملية الادخال والازالة غير مكلفة فقط علينا تحديث العقد المجاورة لاضافة او ازالة عقدة دون الحاجة الى نقل العناصر وأيضا يمكن التنقل بسهولة من بداية القائمة الى نهايتها او العكس بسهولة
(Insertion and removal are inexpensive just update the links in the surrounding nodes
Linear traversal is inexpensive)
- ولكن الوصول العشوائي في القائمة مكلف حيث يحتاج الى بدء التنقل من البداية والتقدم عبر العقد حتى نصل الى العنصر المطلوب وأيضا يمكننا البدء من بداية القائمة والتجول في كل عقدة وهذه تعد طريقة فعالة للتنقل بين العناصر
(Start from beginning or end and traverse each node while counting Linked list allows you to add data dynamically on run time The list is sequential, so access it that way)

ملاحظة هامة: ال linked list المستخدمة في نوعها double حصرًا

↪ أنواع ال Linked list :

1. Single linked list

كل node فيها يؤشر على ال node الذي يليها فقط وال node الأخير لا تؤشر على شيء (null) .

2. 2-Double LinkedList :

كل node فيها يؤشر على ال node الذي يليه وال node الذي قبله .

3. Circle linked list

ذاتها ال Single LinkedList ولكن ال node الأخير لا يؤشر على null ولكن يؤشر على ال node الأول في ال LinkedList .

تعامل ال LinkedList مع List Iterator :

LinkedList تستفيد من List Iterator بشكل فعال نظراً لتصميمها القائم على العقد باستخدام بعض التوابع ، مما يجعل العمليات مثل الإضافة والحذف أسرع في مواقع محددة مقارنة بـ Array List
(ListIterator knows about position use add() from ListIterator to add at a position use remove() from ListIterator to remove at a position LinkedList knows a few things too)

Method in linked list ↩

void addFirst(Object o),	تضيف عنصر في اول كائن ال LinkedList
void addLast(Object o)	تضيف عنصر في اخر كائن ال LinkedList
Object getFirst()	ترجع اول عنصر موجود في كائن ال LinkedList ترمي الاستثناء NoSuchElementException في حال كان كائن ال LinkedList فارغ
Object getLast()	ترجع اخر عنصر موجود في كائن ال LinkedList ترمي الاستثناء NoSuchElementException في حال كان كائن ال LinkedList فارغ
Object removeFirst()	تحدف اول عنصر موجود (كائن LinkedList) وترمي الاستثناء NoSuchElementException في حال كان كائن ال LinkedList فارغ
Object removeLast()	تحدف اول عنصر موجود (كائن LinkedList) وترمي الاستثناء NoSuchElementException في حال كان كائن ال LinkedList فارغ

الفرق بين Array list و LinkedList

Array List

Random access وصول عشوائي

High cost insert and delete
كلفة الإدخال والإخراج عاليةHigh cost random access
كلفة الوصول العشوائي عاليةArray list resizes if needed to
حجمها يتغير حسب الحاجةLinkedList

sequential access وصول متسلسل

low cost insert and delete
كلفة الإدخال والإخراج قليلةHigh cost random access
كلفة الوصول العشوائي عالية

الفرق بين Array list و Vector

Array List

ليس متزامن بشكل افتراضي أي إذا تم استخدامه من قبل عدة Threads في نفس الوقت قد تحدث مشاكل تتعلق بسلامة البيانات

Vector

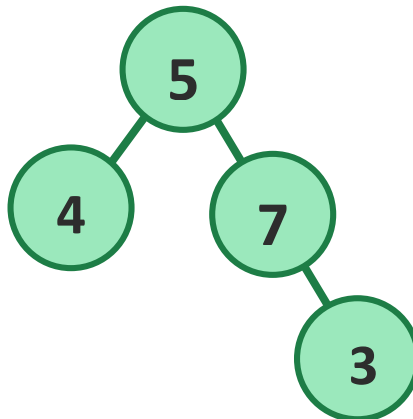
متزامن بشكل افتراضي أي ان جميع العمليات يتم تنفيذها على كائنات محمية من الوصول المتزامن (synchronized Threads)

ملاحظات :

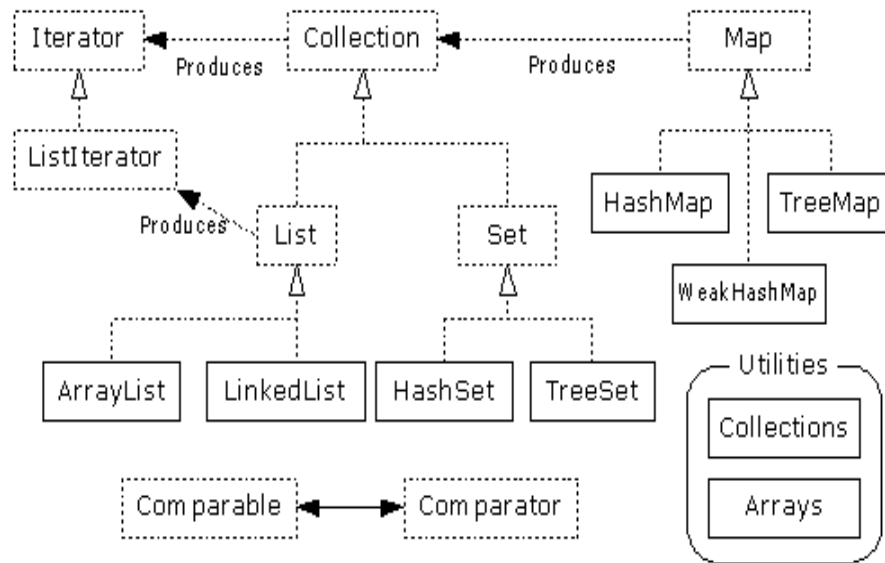
- ★ مهما كان نوع ال DS سواء كان Graph/ Tree يمكننا تخزينها اما ب ArrayList او LinkedList
- ★ يمكننا تخزين ال Graph في مصفوفة متجاورة ثنائية البعد وتتمثل العقد المترابطة بالقيم غير الصفرية والغير مترابطة بالقيم الصفرية أو بعدم وجود قيم

مثال :

- في هذا المثال لدينا Tree سيتم تخزينها في مصفوفة بحيث ان الابن اليساري $2i$ والابن اليميني $2i+1$ ونبدأ من الدليل (0) لانخزن فيه شيء ونتابع بحيث يكون الاب في الدليل (1) اما الابن اليساري فهو 2 لان $2 = 1 * 2$ فيخزن في الخانة ذات الدليل (2) والابن اليميني $3 = 1 + 1 * 2$ فيخزن في الخانة ذات الدليل (3) اما ابن الابن اليميني فيخزن في الخانة ذات الدليل 5 حيث $5 = 1 + 2 * 2$
- ال Tree هي عبارة عن LinkedList فيها مؤشر على الابن اليساري ومؤشر على الابن اليميني
- اذا كانت ال Tree متوازنة (يتم توزيع العقد بشكل متساوي) فان التعقيد هو $O(\log n)$
- اما اذا كانت غير متوازنة (انحراف في التوزيع) فان التعقيد $O(n)$



Map interface (Hash Table)



هذا الانترفيس لا يرث من الانترفيس collection لكنها توفر طرق تخزين لا تتوافر في الاطار collection وبعضها يستخدم للتعامل مع الكلاسات الموجودة في الاطار collection.

يستخدم لتخزين العناصر بشكل key/value أي تخزين القيمة والمفتاح على شكل ازواج (Stores key /value pairs)

ويمكن للمفاتيح المختلفة ان تحتوي على نفس القيمة (Values do not have to be unique)

Collision : tow value have same key

■ مثال على تعريف الـ Map:

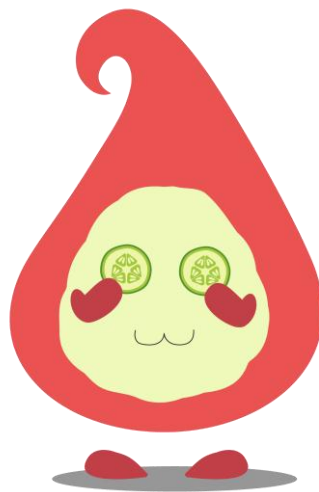
```
Map<String, Integer> ages = new HashMap<>();
```

Map تستخدم لربط كل مفتاح بقيمة معينة (Maps from the key to the value) ولا يمكن أن يكون هناك مفتاحان متطابقان في نفس الـ Map وكل مفتاح يجب أن يكون فريداً ويمكن لكل مفتاح أن يرتبط بقيمة واحدة فقط (Each key in a Map can only be associated with one value, and no key can appear more than once)

:Method in map ↩

Object put(Object key, Object value)	تستخدم لإضافة زوج مفتاح وقيمة
Object get(Object key)	تستخدم لإرجاع القيمة المرتبطة بمفتاح معين
Object remove(Object key)	تستخدم لإزالة العنصر
Int size()	تستخدم لإرجاع عدد العناصر
boolean containsKey(Object key)	تستخدم للتحقق من وجود مفتاح محدد
boolean containsValue(Object value)	تستخدم للتحقق من وجود قيمة محددة
boolean isEmpty()	تستخدم للتحقق من وجود مفتاح محدد

ملاحظة : اذا كان لدينا Data ونريد تخزينها في ال memory حيث يكون بتعقيده (1) ونبحث عن عنصر بزمان ثابت بغض النظر عن عدد العناصر افضل طريقة لتخزينها ما يسمى بال Map بحيث يتم الربط بين key/value



Happiness is not something ready-made.
It comes from your own action.

مثال: نريد كتابة كود بحيث يكون لدينا الاعداد وباقي قسمتها على العدد 10:

الشرح:

تم انشاء ال HashTable بحيث تكون القيمة هي العدد ونخزن بالkey باقي القسمة على 10 بحيث إذا تم ادخال رقم للبحث عنه يتم البحث بالkey بحيث يكون اسهل مثال عند البحث عن عن باقي قسمة العدد 128 على 10 لا داعي للبحث في كل المصفوفة فقط نبحث في عن ال 8 فيكون التعقيد $O(1)$ في احسن الأحوال . اما في أوسط الحالات فان التعقيد $O(n/m)$ أي العدد الكلي على عدد التصادمات واسوأ الحالات $O(n)$

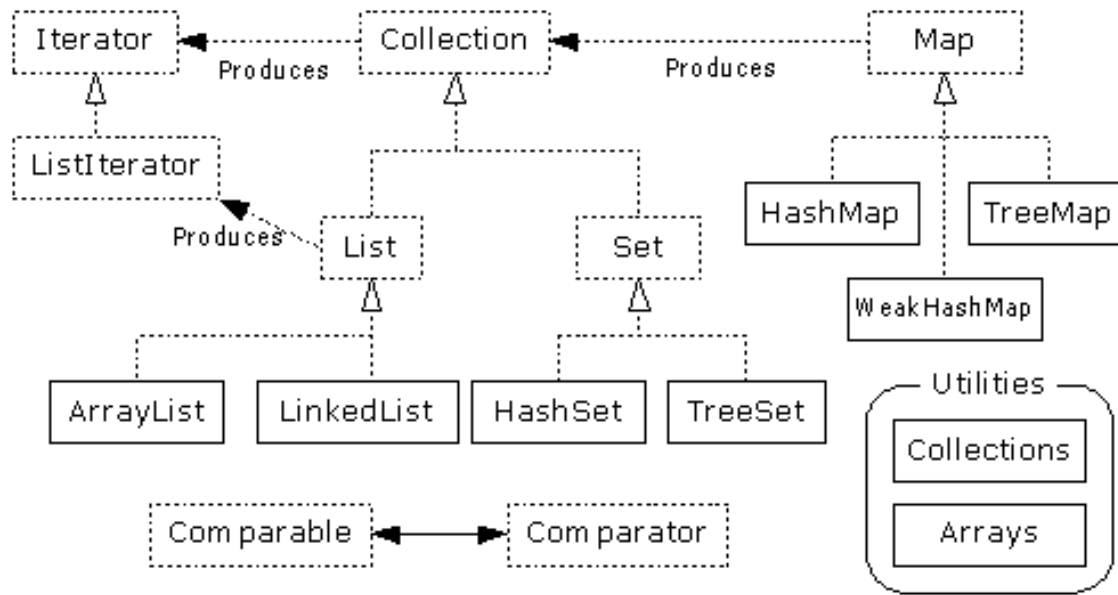
في حال كان البحث أيضا عن العدد 68 فيصبح هناك اكثر من key لها نفس الاسم فنلجأ الى التخزين في LinkedList

في ال java8 تم التطوير بحيث يمكن التخزين في عن طريق BTS (فأصبح تعقيد العمليات $O(\log n)$)

HashMap and TreeMap Context

```
1 import java.util.HashMap;
2 import java.util.Map;
3 public class Test {
4     public static void main(String[] args) {
5         // تعريف مصفوفة من الأعداد
6         int[] numbers = {128, 25, 37, 40, 55, 62, 78};
7         // إنشاء خريطة لتخزين الأعداد وباقي القسمة على 10
8         Map<Integer, Integer> remainderMap = new HashMap<>();
9         // ملء الخريطة
10        for (int number : numbers) {
11            remainderMap.put(number, number % 10);}
12        // طباعة النتائج
13        for (Map.Entry<Integer, Integer> entry : remainderMap.entrySet()) {
14            System.out.println("عدد: " + entry.getKey() + ", باقي القسمة على 10: " +
15                entry.getValue());
16        }
17    }
```

بعض الكلاسات التي قامت بـ implements لـ Map interface



- الكلاس HashMap يستخدم لتخزين العناصر بشكل key/value حيث يتم إعطاء مفتاح لكل قيمة يتم تخزينها بداخل الـ hashmap و يجب ان تكون كل قيمة يتم تخزينها في الكائن معطاة لـ key غير مستخدمة من قبل ويحتوي هذا الكلاس على العديد من الميثود
- الكلاس TreeMap يستخدم لتخزين العناصر بشكل key/value بالإضافة الى ترتيب هذه العناصر في الذاكرة بشكل تصاعدي (حسب الـ key) أيضا يجب ان تكون كل قيمة يتم تخزينها في الكائن معطاة لـ key غير مستخدمة من قبل

الفارق بين Tree Map و HashMap

Tree Map

يحافظ على ترتيب العناصر

يقدم تكلفة زمنية لوغاريتمية ($\log(n)$) للعمليات الأساسية مثل الإضافة والحذف والبحث بسبب استخدامه لشجرة ثنائية متوازنة

مناسب عندما تحتاج إلى تخزين الأزواج (مفتاح-قيمة) بطريقة مرتبة وعمليات على مجموعة مرتبة مثل البحث عن نطاقات من المفاتيح

HashMap

لا يضمن ترتيب العناصر

يقدم أداءً زمنياً ثابتاً $O(1)$ بشكل عام للعمليات الأساسية مثل الإضافة والحذف والبحث

مناسب عندما تحتاج إلى تخزين الأزواج (مفتاح-قيمة) والوصول السريع إليها ولا يهتمك ترتيب العناصر

Hashtable :

The java.util.Hash table class is a class in Java that provides a key-value data structure similar to the Map interface. It was part of the original Java Collections framework and It is similar to HashMap introduced in Java 1.0 It is similar to HashMap, but is synchronized

هو كلاس ينتمي إلى حزمة java.util ويستخدم لتخزين البيانات في شكل أزواج من المفتاح والقيمة ويقوم ب Java Collections Map ل implements ويعتبر جزءاً من مكتبة

■ تعريف ال Hashtable:

```
Hashtable<String, Integer> hashtable = new Hashtable<>();
```

يعتبر Hashtable خيطياً (Thread-safe) بشكل افتراضي، مما يعني أنه يمكن استخدامه في بيئات متعددة ال threads.

كيفية تخزين البيانات في Hashtable

- عندما تضيف عنصراً إلى ال Hashtable، يتم تمرير المفتاح إلى دالة التجزئة hashCode().
- تقوم دالة التجزئة بتحويل المفتاح إلى رقم صحيح (hash code) وثم إعادته حيث يستخدم لتحديد موضع الدليل (bucket) الذي سيتم تخزين الزوج مفتاح_قيمة (key _ value) فيه.

✎ كيف يمكن تحديد الموضع بشكل دقيق (Bucket Index)

أولاً يتم تقسيم الجدول إلى مجموعة من الدلائل (buckets)، وكل دليل يمكن أن يحتوي على عنصر أو عدة عناصر. يستخدم ال Hashtable القيمة التي تعيدها دالة التجزئة hashCode() لتحديد أي دليل سيحتوي على العنصر. الموضع يتم حسابه بالعملية البتية (Bitwise & Operation) أو (Bitwise AND Operation) هكذا:

```
String key = "exampleKey";
int hashCode = key.hashCode();
int bucketIndex = hashCode & (table.length - 1);
```

حيث طول جدول التجزئة (table.length) هو 16:
العملية البتية (Bitwise & Operation) تجعل
الموضع محصوراً بين 0 و 15.
 $bucketIndex = hashCode \& 15$

```
String key = "hi";
int hashCode = key.hashCode();
```

مثلاً لفترض أنك تريد تخزين المفتاح "hi" مع القيمة 5
فيكون ال hashCode(5)=00000101
وطول جدول التجزئة (15)=00001111
ف عندئذ يكون $00000101 = 00000101 \& 00001111$

الكود كاملاً

```

1 import java.util.Hashtable;
2 public class test {
3     public static void main(String[] args) {
4         Hashtable<String, Integer> hashtable = new Hashtable<>();
5         // مع القيمة 5 "hi" تخزين المفتاح
6         hashtable.put("hi", 5);
7         // عرض القيمة المرتبطة بالمفتاح "hi"
8         System.out.println("Value for 'hi': " + hashtable.get("hi"));
9     }
10 }

```

ملخص لما سبق :

الميزة	الخاصية	Hash table	Map (مثل Hash map و Tree map)
التزامن (thread-Safety)	متزامنة مما يجعلها آمنة لاستخدامها مع عدة خيوط (thread)	ليست مترابطة بشكل افتراضي (مثل hashmap)	
الأداء	أبطأ بسبب التزامن	أسرع بسبب عدم وجود تزامن افتراضي	
دعم القيم null	لا تسمح بالقيم null سواء للمفتاح أو القيمة	Hash map تسمح بمفتاح واحد null و عدة قيم null	
إضافة منذ java	موجودة منذ الإصدار الأول من java	الواجهة map وأغلب تطبيقاتها أضيفت لاحقاً (مثل hash map في java 1.2)	
الترتيب	لا تحافظ على ترتيب الإدخال	يعتمد على التطبيق (Tree map مرتبة ، hash map غير مرتبة)	

ملاحظة: التصادم (Collision) يحدث في هياكل البيانات مثل Hash Table عندما تقوم دالة التجزئة (Hash Function) بإرجاع نفس المؤشر (Index) لمفتاحين مختلفين. بمعنى آخر، يتم تعيين مفتاحين (Keys) إلى نفس الموضع في الجدول (Bucket) داخل جدول التجزئة

فائدة اربسيزية :

Binary Tree Search (البحث في الشجرة الثنائية): هي هيكل بيانات يتم فيه تنظيم العناصر بطريقة (Binary Search Tree - BST) تجعل عمليات البحث، الإدراج، والحذف فعالة.

1. يتم ترتيب العقد بحيث تكون القيم في العقدة اليسرى أصغر من الجذر.

2. القيم في العقدة اليمنى أكبر من الجذر

3. تستخدم من أجل : البحث السريع عن القيم.

4. تخزين البيانات بترتيب معين

5. هيكلية البيانات لتقليل تعقيد العمليات

Bulk Operations

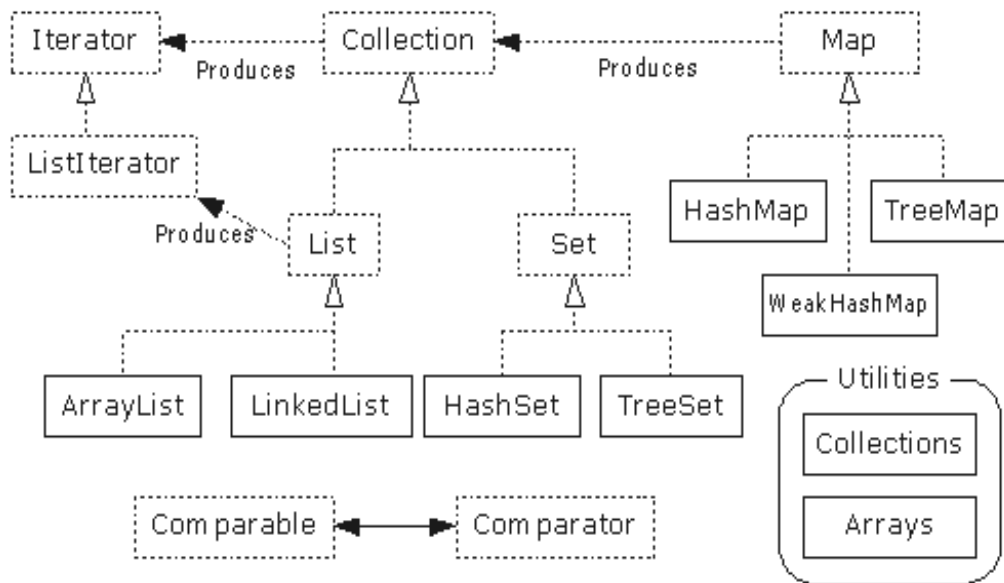
In addition to the basic operations, a Collection may provide “bulk” operations

ف ال تسمح لنا بالتعامل مع مجموعة من العناصر دفعة واحدة مما يسهل إدارة البيانات بشكل اكبر

Methods ↙

!boolean containsAll(Collection c)	هذه الدالة تتحقق اذا كانت المجموعة الحالية تحتوي على جميع العناصر الموجودة في مجموعة اخرى
!boolean addAll(Collection c)	تضيف جميع العناصر من مجموعة معينة الى المجموعة الحالية
!boolean removeAll(Collection c)	تقوم بإزالة جميع العناصر الموجودة في المجموعة الحالية
!boolean retainAll(Collection c)	تحتفظ فقط بالعناصر الموجودة في المجموعة الحالية والتي تتواجد أيضا في المجموعة الأخرى (تقاطع) مما يعني انها تقوم بإزالة أي عناصر غير موجودة في المجموعة المحددة
()void clear	إزالة جميع العناصر من المجموعة مما يجعلها فارغة
()Object[] toArray	تقوم بتحويل العناصر في المجموعة الى مصفوفة من Object نوع
()Object[] toArray(Object a[])	تحويل العناصر في المجموعة الى مصفوفة من النوع المحدد

Utilities Context :



The Collections class provides a number of static methods for fundamental algorithms
Most operate on Lists, some on all Collections

الكلاس يحتوي على دوال ثابتة هذه الدوال تساعد كثيرا في الوصول الى العناصر والتلاعب بها سواء كانت تتعامل مع الكلاسات التي ترث من ال وتسمى هذه المجموعة من الخوارزميات: algorithms collection

أمثلة :

public static void Copy	تنسخ عناصر الكائن في آخر الكائن
Fill public static void	تضع الكائن كقيمة لجميع عناصر الكائن
public static void Reverse	تعكس ترتيب عناصر الكائن list
Public int Max	ترجع اكبر عنصر موجود في الكائن
Shuffle public static void	تقوم بتبديل اماكن عناصر الكائن بشكل عشوائي مرة واحدة
public static void Sort	ترتيب العناصر الموجودة في الكائن list

✦ Wrappers :

هي ال class التي قامت ب implementations تخصيص سلوك المجموعة دون الحاجة الى التعديل على المجموعة الاصلية وهناك نوعان :

1. synchronized Collections, Lists, Sets, etc. :

نحتاج الى مزامنة كائن مستخدم في تخزين البيانات في حال كان البرنامج يحتوي على عدة أوامر تعمل بشكل متزامن (في حال وجود thread)

طريقة المزامنة:

انشاء الكائن باستخدام دوال التزامن الموجودة في كلاس Collections الملائمة لنوعه

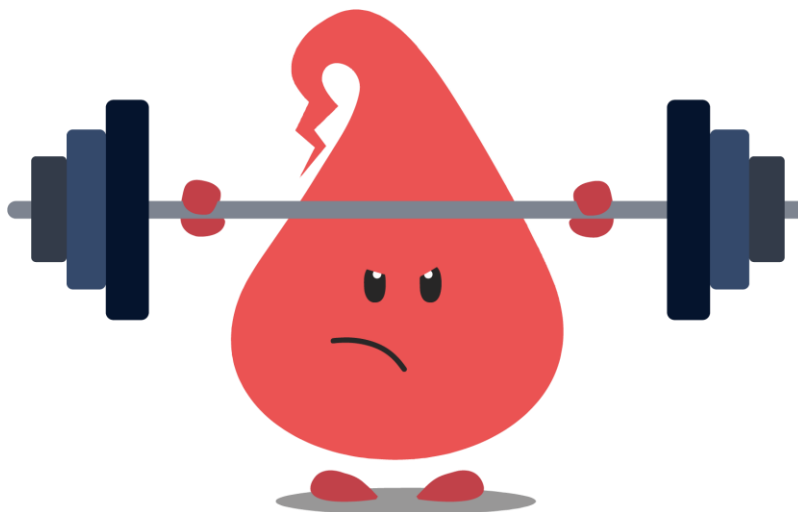
استخدام الكلمة المحجوزة synchronized

2. unmodifiable Collections, Lists, Sets, etc. :

تستخدم لتجميد المجموعة وجعلها لا تقبل أي تعديل مثل إضافة او حذف عناصر وهذا ما يجعل

لمجموعة محمية من التعديلات الغير مقصودة

The two hardest tests on the road are the patience to wait for the right moment and the courage not to be disappointed with what we encounter.



The End